

Guía Práctica y Detallada: Estructura de Proyectos y Gestión de Paquetes en Java

Modulo 01: Fundamentos Modernos de Programación con Java

Mayo 2026

1. Introducción

Cuando una persona comienza a programar en Java, normalmente crea archivos sueltos y ejecuta comandos sin comprender realmente qué está ocurriendo detrás del proceso. Sin embargo, en el desarrollo profesional, Java trabaja bajo una filosofía muy organizada.

Un proyecto Java no es simplemente “un archivo que corre”. Es una estructura completa donde cada carpeta tiene un propósito específico:

- Separar el código fuente.
- Separar el código compilado.
- Mantener el orden del proyecto.
- Facilitar el trabajo en equipo.
- Permitir mantenimiento a largo plazo.

Esta guía busca que entiendas:

1. Cómo se organiza un proyecto Java.
2. Qué hace realmente el compilador.
3. Qué significa `package`.
4. Cómo Java encuentra una clase.
5. Cómo ejecutar programas correctamente desde terminal.

2. La Arquitectura de un Proyecto Java

2.1. ¿Por qué organizar un proyecto?

Imagina un edificio de apartamentos sin números, sin pisos y sin nombres. Sería imposible encontrar dónde vive cada persona.

Lo mismo ocurre en programación:

- Sin organización, los archivos se vuelven imposibles de mantener.
- Dos clases pueden tener el mismo nombre.
- El compilador no sabrá dónde buscar.
- El trabajo en equipo se vuelve caótico.

Por eso Java utiliza una estructura profesional basada en carpetas y paquetes.

2.2. Las Carpetas Fundamentales

Antes de escribir una sola línea de código, debemos construir la arquitectura básica del proyecto.

2.2.1. Carpeta `src`

Carpeta Source

La carpeta `src` significa **source** (fuente).
Aquí vive el código que el programador escribe manualmente.

Dentro de `src` guardamos:

- Archivos `.java`
- Clases
- Interfaces
- Paquetes

Ejemplo:

```
src/  
  Calculadora.java
```

2.2.2. Carpeta `classes`

Carpeta Compilada

La carpeta `classes` almacena los archivos compilados `.class`.

Estos archivos:

- Ya no son código humano.
- Son Bytecode.
- Son instrucciones para la JVM.

Importante: Nunca modificamos manualmente un archivo `.class`.

Ejemplo:

```
classes/  
Calculadora.class
```

2.2.3. Carpeta test

Pruebas

La carpeta `test` se usa para verificar automáticamente que el software funciona.

Aquí se escriben:

- pruebas unitarias
- validaciones automáticas
- simulaciones

Ejemplo:

```
test/  
CalculadoraTest.java
```

3. ¿Qué es la Compilación?

3.1. El problema

Las computadoras no entienden Java directamente.

Elas entienden:

- código máquina
- instrucciones binarias
- operaciones de bajo nivel

Entonces necesitamos un traductor.

3.2. El compilador javac

Java utiliza el compilador:

`javac`

Su trabajo es:

1. Leer archivos `.java`
2. Verificar errores
3. Transformarlos en Bytecode
4. Generar archivos `.class`

3.3. Flujo completo

`Archivo.java` → Compilador `javac` → `Archivo.class` → JVM → Programa ejecutándose

4. El Concepto de Package

4.1. ¿Qué es un Package?

Un package es un sistema de organización lógica.

Definición

Un **package** funciona como un espacio de nombres que evita conflictos entre clases.

4.2. Problema real sin packages

Imagina dos desarrolladores:

- Ana crea:

`User.java`

- Carlos también crea:

`User.java`

¿Cómo sabe Java cuál usar?

La respuesta es: **usando packages.**

4.3. Ejemplo correcto

```
1 package com.empresa.seguridad;  
2  
3 public class User {  
4  
5 }
```

y otro:

```
1 package com.empresa.recursoshumanos;  
2  
3 public class User {  
4  
5 }
```

Ahora Java sí puede diferenciarlos.

5. La Relación Entre Package y Carpetas

En Java:

El package DEBE coincidir con la estructura de carpetas.

Ejemplo:

```
1 package com.mycompany.calculadora;
```

Esto significa que Java espera:

```
com/  
  mycompany/  
    calculadora/
```

5.1. ¿Quién crea esas carpetas?

El compilador puede crearlas automáticamente usando:

-d

6. Práctica Guiada: Primera Aplicación Java

6.1. Objetivo

Aprenderás:

- crear un proyecto
- compilar correctamente
- usar packages
- ejecutar desde terminal

6.2. Paso 1: Crear el Proyecto

En el escritorio crea:

Laboratorio1

Dentro:

Laboratorio1/
src/
classes/
test/

6.3. ¿Por qué crear estas carpetas desde el inicio?

Porque estamos simulando una estructura profesional.

- `src` = código humano
- `classes` = código compilado
- `test` = validaciones

6.4. Paso 2: Abrir VS Code

Abrir:

File > Open Folder

Seleccionar:

Laboratorio1

6.5. ¿Por qué abrir la carpeta raíz?

Porque VS Code necesita entender:

- toda la estructura
- las rutas relativas
- la terminal integrada

6.6. Paso 3: Crear el Archivo Java

Dentro de:

src

crear:

Calculadora.java

Código (*Copia el código no hay necesidad de entenderlo todo todavía*):

```
1 package com.mycompany.calculadora;
2
3 public class Calculadora {
4
5     public static void main(String[] args) {
6
7         System.out.println("---Ejecutando --Calculadora---");
8
9         double n1 = 10.5;
10        double n2 = 5.2;
11
12        System.out.println("Resultado:" + (n1 + n2));
13    }
14 }
```

7. Análisis Línea por Línea

7.1. La Línea package

```
1 package com.mycompany.calculadora;
```

Le dice a Java:

“Esta clase pertenece al espacio de nombres
com.mycompany.calculadora”

7.2. La Clase

```
1 public class Calculadora
```

Define una clase pública llamada:

Calculadora

7.3. Método main

```
1 public static void main(String[] args)
```

Es el punto de entrada del programa.
La JVM comienza a ejecutar desde aquí.

7.4. System.out.println

```
1 System.out.println("Texto");
```

Imprime texto en consola.

7.5. Variables double

```
1 double n1 = 10.5;
```

double almacena números decimales.

8. Compilación desde Terminal

8.1. Abrir la Terminal

En VS Code:

Ctrl + ‘

8.2. Comando de Compilación

Windows:

```
javac -d classes src\Calculadora.java
```

Linux/Mac:

```
javac -d classes src/Calculadora.java
```

9. Descomposición del Comando

9.1. javac

Es el compilador oficial de Java.

9.2. -d classes

-d significa:

destination

Le indica al compilador:

“Guarde los archivos compilados en la carpeta classes”

9.3. src\Calculadora.java

Es la ruta del archivo fuente.

9.4. ¿Qué ocurre internamente?

El compilador:

1. lee el package
2. crea carpetas automáticamente
3. genera el archivo class

Resultado:

```
classes/  
  com/  
    mycompany/  
      calculadora/  
        Calculadora.class
```

10. Ejecución del Programa

10.1. Comando Correcto

```
java -cp classes com.mycompany.calculadora.Calculadora
```

11. Análisis del Comando java

11.1. java

Inicia la JVM.

11.2. -cp classes

cp significa:

classpath

Le dice a Java:

“Busca las clases dentro de la carpeta classes”

11.3. Nombre Completo

```
1 com.mycompany.calculadora.Calculadora
```

No es solo el nombre de la clase.

Es:

package + clase

La JVM necesita la ruta completa.

12. Error Común

Si escribes:

```
java -cp classes Calculadora
```

Obtendrás algo similar a:

```
Error: Could not find or load main class Calculadora
```


13. ¿Por Qué Ocurre el Error?

Porque Java busca:

```
classes/Calculadora.class
```

Pero realmente existe en:

```
classes/com/mycompany/calculadora/Calculadora.class
```

Por eso debes usar:

```
com.mycompany.calculadora.Calculadora
```

14. Ejercicio de Refuerzo 1

14.1. Objetivo

Practicar packages diferentes.

14.2. Crear Archivo

Dentro de:

```
src
```

crear:

```
Saludador.java
```

14.3. Código

```
1 package org.universidad.util;
2
3 public class Saludador {
4
5     public static void main(String[] args) {
6
7         System.out.println("Un--gran--saludo--para--la┐comunidad--de--
8             DevSenior--Code");
9     }
10 }
```

14.4. Compilar

Windows:

```
javac -d classes src\Saludador.java
```

Linux/Mac:

```
javac -d classes src/Saludador.java
```

14.5. Resultado Esperado

```
classes/  
  org/  
    universidad/  
      util/  
        Saludador.class
```

15. Ejercicio de Refuerzo 2

Intentar ejecutar:

```
java -cp classes Saludador
```

15.1. Resultado

Error:

```
Could not find or load main class Saludador
```

15.2. Corrección

```
java -cp classes org.universidad.util.Saludador
```

16. Conclusiones

Después de esta práctica debes comprender que:

- Java trabaja con estructuras organizadas.
- Los packages no son decoración.
- El compilador genera carpetas automáticamente.
- La JVM necesita el nombre completo de la clase.
- `-d` controla dónde se guarda el código compilado.
- `-cp` le dice a Java dónde buscar las clases.

Idea Fundamental

Cuando usas packages, el nombre verdadero de una clase deja de ser solo:
`Calculadora`
y pasa a ser:
`com.mycompany.calculadora.Calculadora`